

QuickRide

A PROJECT REPORT

Submitted by

PATEL BHAVY J. (23BECE30169)

PATEL BRIJKUMAR H. (23BECE30170)

PATEL DARVIN V. (23BECE30171)

PATEL MEET PINTU (23BECE30288)

In fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

<Name of the Department>



LDRP Institute of Technology and Research, Gandhinagar

Kadi Sarva Vishwavidyalaya

NOVEMBER, 2026

***LDRP INSTITUTE OF TECHNOLOGY AND
RESEARCH
GANDHINAGAR***

CE-IT Department



CERTIFICATE

This is to certify that the Project Work entitled “**QuickRide**” has been carried out by **Patel Bhavy J. (23BECE30169)** under my guidance in fulfilment of the degree of Bachelor of Engineering in Computer Engineering Semester-7 of Kadi Sarva Vishwavidyalaya University during the academic year 2025-2026.

KADI SARVA VISHWAVIDYALAYA

Prof. Avani Patel

Internal Guide

LDRP-ITR

Dr. Ashishkumar Patel

Head of the Department

LDRP-ITR

***LDRP INSTITUTE OF TECHNOLOGY AND
RESEARCH
GANDHINAGAR***

CE-IT Department



CERTIFICATE

This is to certify that the Project Work entitled “**QuickRide**” has been carried out by **Patel Brijkumar H. (23BECE30170)** under my guidance in fulfilment of the degree of Bachelor of Engineering in Computer Engineering Semester-7 of Kadi Sarva Vishwavidyalaya University during the academic year 2025-2026.

Prof. Avani Patel

Internal Guide

LDRP-ITR

Dr. Ashishkumar Patel

Head of the Department

LDRP-ITR

***LDRP INSTITUTE OF TECHNOLOGY AND
RESEARCH
GANDHINAGAR***

CE-IT Department



CERTIFICATE

This is to certify that the Project Work entitled “**QuickRide**” has been carried out by **Patel Darvin V. (23BECE30171)** under my guidance in fulfilment of the degree of Bachelor of Engineering in Computer Engineering Semester-7 of Kadi Sarva Vishwavidyalaya University during the academic year 2025-2026.

Prof. Avani Patel

Internal Guide

LDRP ITR

Dr. Ashishkumar Patel

Head of the Department

LDRP ITR

***LDRP INSTITUTE OF TECHNOLOGY AND
RESEARCH
GANDHINAGAR***

CE-IT Department



CERTIFICATE

This is to certify that the Project Work entitled “**QuickRide**” has been carried out by **Patel Meet Pintu (23BECE30288)** under my guidance in fulfilment of the degree of Bachelor of Engineering in Computer Engineering Semester-7 of Kadi Sarva Vishwavidyalaya University during the academic year 2025-2026.

KADI SARVA VISHWAVIDYALAYA

Prof. Avani Patel

Internal Guide

LDRP-ITR

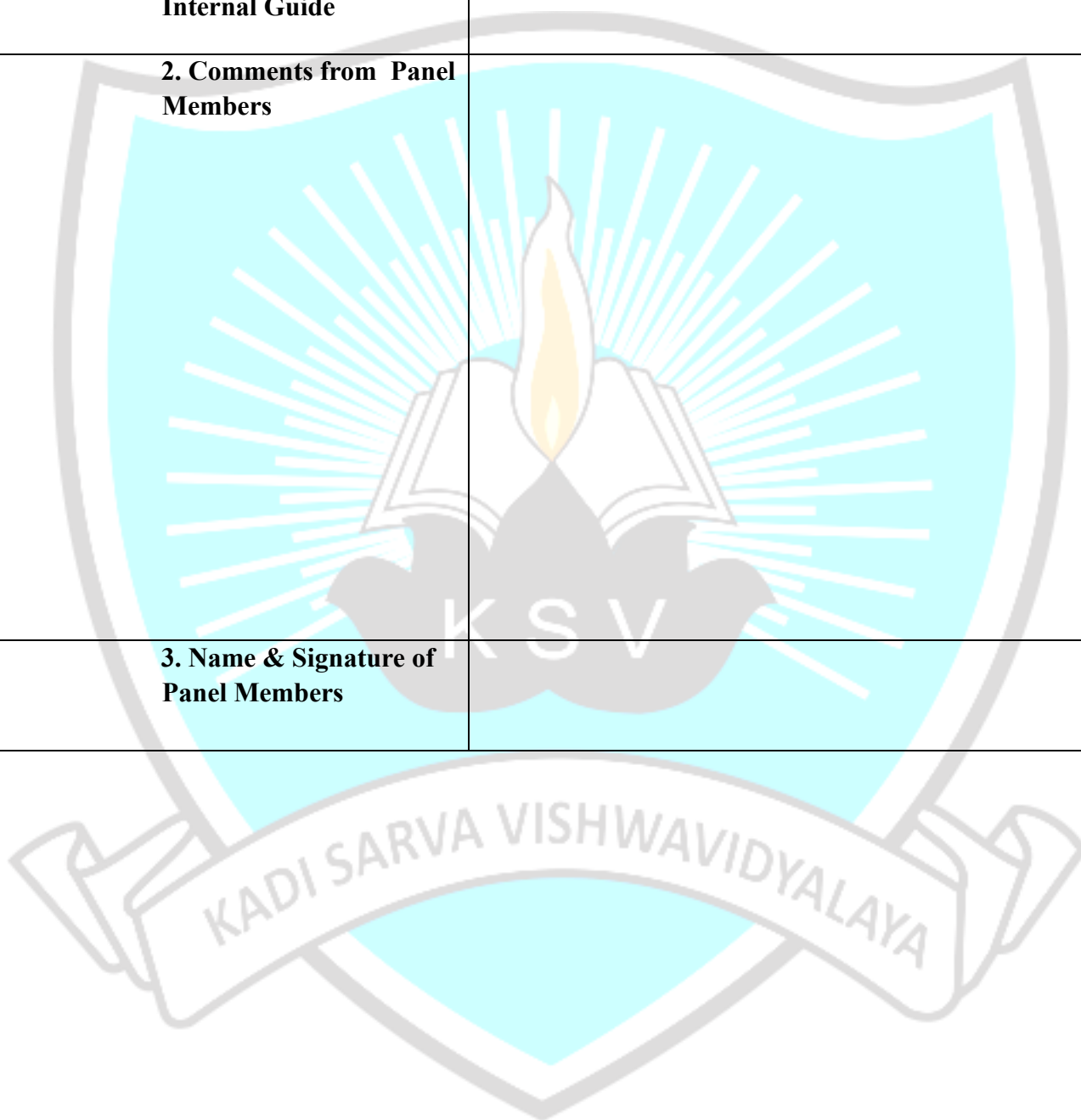
Dr. Ashishkumar Patel

Head of the Department

LDRP-ITR

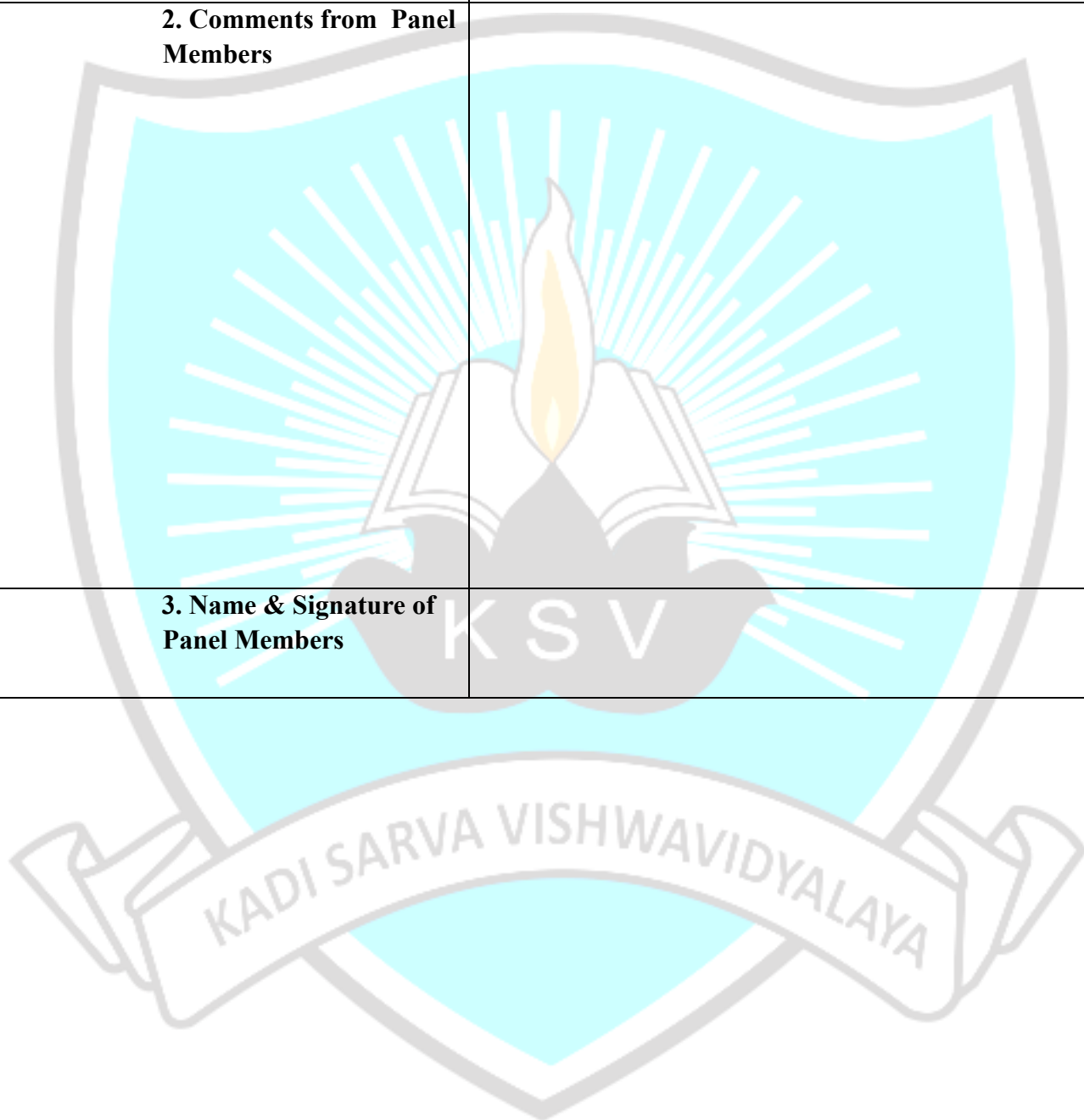
Presentation-I for Project-II

1. Name & Signature of Internal Guide	
2. Comments from Panel Members	
3. Name & Signature of Panel Members	



Presentation-II for Project-II

1. Name & Signature of Internal Guide	
2. Comments from Panel Members	
3. Name & Signature of Panel Members	



ACKNOWLEDGEMENT

We are grateful to our internal guide **Prof. Avani Patel** for providing us valuable guidance for the PROJECT and other domain related information.

Also, our hearty gratitude to Head of Department, **Dr. Ashishkumar Patel** for giving us an opportunity for learning through the process of project we by which we can improve our technical skills along with practical exposure and its applicability in the industry.

PATEL BHAVY J.
(23BECE30169)

PATEL BRIJKUMAR H.
(23BECE30170)

PATEL DARVIN V.
(23BECE30171)

PATEL MEET PINTU
(23BECE30288)



ABSTRACT

The **QuickRide Ride Booking System** is a full-stack MERN-based web application developed to provide a secure, efficient, and real-time platform for booking rides between passengers and drivers. The primary objective of this project is to simplify the ride-booking process by enabling users to book rides online, track drivers in real time, calculate fares automatically, and complete secure online payments. The system aims to reduce manual effort, improve transportation services, and enhance the overall user experience.

In this system, users can register and log in securely using **JWT (JSON Web Token)** authentication, while passwords are protected using **bcrypt** encryption. After successful authentication, users can enter their pickup and destination locations, view the estimated fare, and request a ride. The application uses the **OpenRouteService Maps API** to calculate travel distance, estimated travel time, and route information. The backend then identifies nearby available captains and sends ride requests through **Socket.IO**, enabling instant real-time communication.

Captains can register their vehicle details, manage their online or offline availability, receive ride requests, accept or reject bookings, share live location, start and complete rides, and monitor their earnings through a dedicated dashboard. The system also maintains ride history, allows users to rate and review completed rides, and automatically updates the captain's average rating based on customer feedback.

The application is developed using **React.js**, **Vite**, and **Tailwind CSS** for the frontend, while **Node.js**, **Express.js**, and **MongoDB Atlas** are used for backend development and cloud database management. Additional technologies such as **Socket.IO** enable real-time ride updates, and **Razorpay** is integrated to support secure online payment processing. The frontend is deployed on **Vercel**, and the backend is hosted on **Render**, ensuring high availability and scalability.

By automating ride booking, driver matching, fare calculation, payment processing, and live tracking, the **QuickRide Ride Booking System** provides a modern, reliable, and user-friendly transportation solution. The project demonstrates the practical implementation of the MERN stack along with real-time communication technologies, offering a secure and scalable platform suitable for future enhancements and deployment in real-world ride-sharing services.

TABLE OF CONTENTS:

Acknowledgement	i
Abstract	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Introduction	2
1.2 Scope	2
1.3 Project summary and Purpose	3
1.4 Overview of the project	3
1.5 Problem definition	4
2 Technology and Literature Review	5
2.1 About Tools and Technology	6
2.2 Brief History of Work Done	6
3 System Requirements Study	8
3.1 User Characteristics	9
3.2 Hardware and Software Requirements	9
3.3 Constraints	10
3.3.1 Regulatory Policies	11
3.3.2 Hardware Limitations	11
3.3.3 Interfaces to Other Applications	11
3.3.4 Parallel Operations	12
3.3.5 Higher Order Language Requirements	12
3.3.6 Reliability Requirements	13
3.3.7 Criticality of the Application	13
3.3.8 Safety and Security Consideration	13

3.4 Assumptions and Dependencies	14
----------------------------------	----

4 System Analysis	15
--------------------------	-----------

4.1 Study of Current System	16
-----------------------------	----

4.2 Problem and Weaknesses of Current System	16
--	----

4.3 Requirements of New System	17
--------------------------------	----

4.3.1 User Requirements	17
-------------------------	----

4.3.2 System Requirements	17
---------------------------	----

4.4 Feasibility Study	18
-----------------------	----

4.4.1 Does the system contribute to the overall objectives of the organization?	18
---	----

4.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints?	19
---	----

4.4.3 Can the system be integrated with other system which are already in place?	19
---	----

4.5 Requirements Validation	19
-----------------------------	----

4.6 Activity/Process in New System (Use event table)	20
--	----

4.7 Class Diagram	21
-------------------	----

4.8 E-R Diagram	22
-----------------	----

4.9 System Activity (Use case and/or scenario diagram)	23
--	----

4.10 Object Interaction	24
-------------------------	----

4.11 State Transition Diagram	25
-------------------------------	----

5 System Design	26
------------------------	-----------

5.1 System Application Design	28
-------------------------------	----

5.1.1 Method Pseudo code	28
--------------------------	----

5.2 Database Design/Data Structure Design	29
---	----

5.2.1 Table and Relationship	29
------------------------------	----

5.2.2 Logical Description of Data	30
-----------------------------------	----

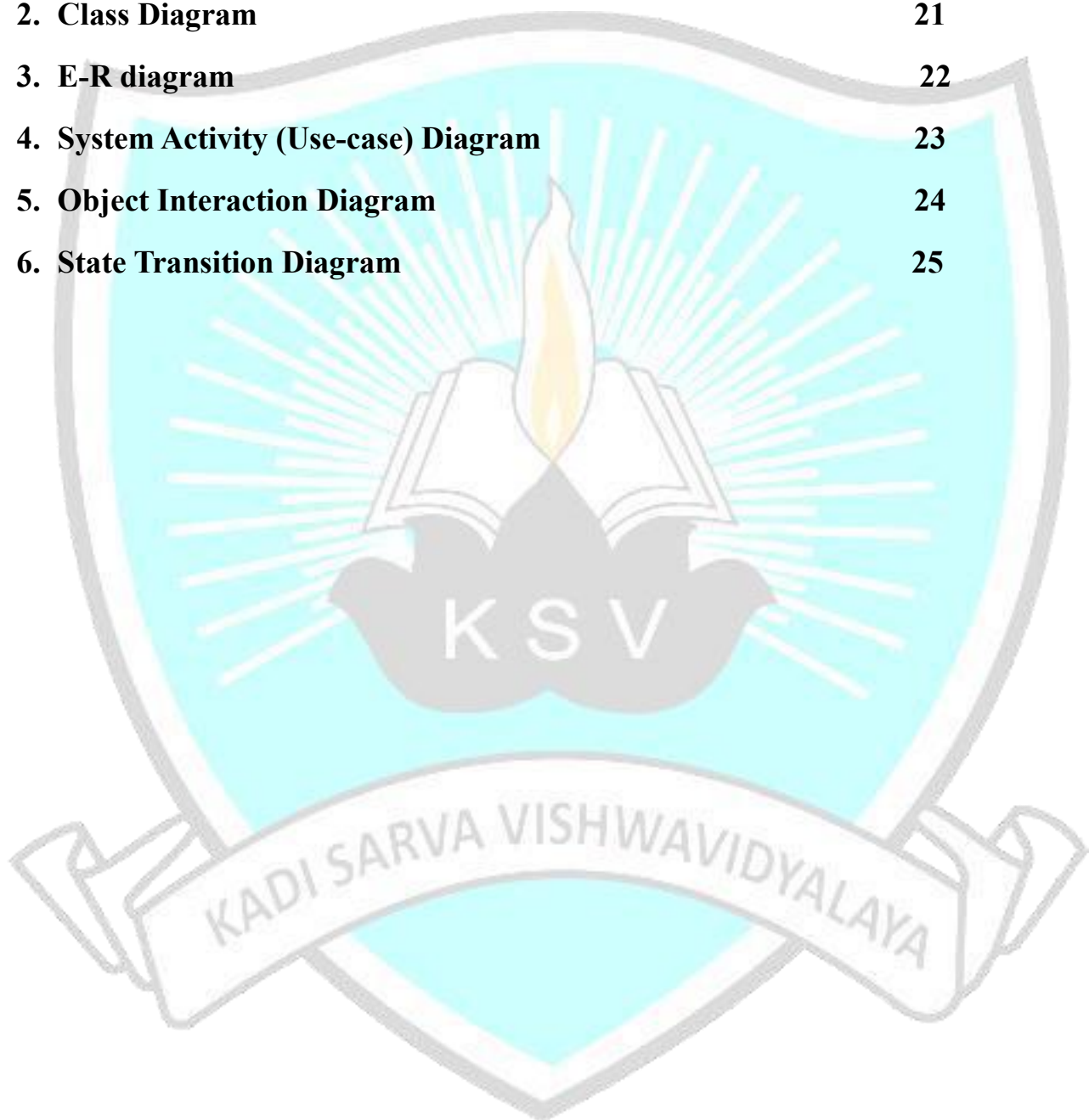
5.3 Input/output and Interface Design	35
---------------------------------------	----

5.3.1 State Transition/UML Diagram	35
------------------------------------	----

5.3.2 Samples Of Forms, Reports and Interface	
---	--

LIST OF FIGURES

1. Activity/Process Diagram	20
2. Class Diagram	21
3. E-R diagram	22
4. System Activity (Use-case) Diagram	23
5. Object Interaction Diagram	24
6. State Transition Diagram	25



LIST OF TABLES

1. Users table	31
2. Collection agent	31
3. Admin table	32
4. Authentication & User Management table	32
5. Features Table	33
6. Route Optimization	33
7. Integration & Performance table	
8. Security & UI table	34



1. INTRODUCTION

1.1. Introduction

1.2. Scope

1.3. Project summary and Purpose

1.4. Overview of the project

1.5. Problem definition



1.1 Introduction

QuickRide is a full-stack MERN-based online ride-booking application that connects passengers with nearby drivers in real time. The system provides a simple, secure, and efficient platform for booking rides, tracking drivers, calculating fares, and completing payments.

The application is developed using modern web technologies including React.js, Node.js, Express.js, MongoDB Atlas, Socket.IO, and OpenRouteService Maps API. QuickRide enables users to register, log in securely, select pickup and destination locations, view fare estimates, and track rides live. Drivers (Captains) can manage their availability, accept ride requests, navigate to passengers, and complete trips while monitoring their earnings.

The system uses JWT authentication, bcrypt password hashing, protected APIs, and token blacklisting to ensure secure access. Socket.IO enables real-time communication between users and captains, allowing instant ride requests, live location sharing, ride status updates, and notifications.

Overall, QuickRide provides a modern transportation solution that improves convenience, reduces waiting time, and delivers a smooth ride-booking experience.

1.2 Scope

The scope of QuickRide includes all major functionalities required for an online ride-booking platform.

The project covers:

- Secure user registration and login
- Captain registration and vehicle management
- Ride booking
- Live driver tracking
- Real-time notifications
- Fare calculation
- Ride history
- Rating and review system
- Earnings dashboard
- Payment integration
- Mobile responsive interface
- Cloud database storage

- Deployment on Vercel and Render

The system is suitable for taxi services, ride-sharing companies, college transportation systems, and local cab service providers.

Future enhancements include AI-based route optimization, ride scheduling, multilingual support, and electric vehicle management.

1.3 Project Summary and Purpose

QuickRide is designed to simplify urban transportation by providing an efficient online ride-booking platform.

Traditional taxi booking often requires phone calls or waiting on the roadside. Users cannot easily know driver availability, estimated fare, or arrival time.

QuickRide eliminates these issues by allowing passengers to book rides directly from their mobile or web browser.

The primary objectives are:

- Reduce ride booking time
- Provide real-time driver matching
- Improve transportation efficiency
- Secure user information
- Provide transparent fare calculation
- Enable real-time communication
- Offer smooth ride tracking
- Improve passenger safety
- Manage driver earnings efficiently

The project demonstrates the implementation of a scalable MERN Stack application with real-time communication using Socket.IO.

1.4 Overview of the Project

QuickRide follows the MERN Stack architecture consisting of:

Frontend

- React.js
- Vite
- Tailwind CSS
- GSAP
- Axios
- Socket.IO Client

Backend

- Node.js
- Express.js
- Socket.IO
- JWT Authentication
- bcrypt
- Express Validator

Database

- MongoDB Atlas

APIs

- OpenRouteService Maps API
- Razorpay Payment API

Hosting

- Vercel
- Render

The application begins with secure registration. After login, passengers select pickup and destination locations. The system calculates distance and fare, identifies nearby available captains, and sends ride requests using Socket.IO.

Drivers receive notifications instantly and may accept or reject the request. Once accepted, both passenger and captain can monitor live ride progress until completion.

After completing the trip, users can rate the driver and submit reviews. Driver earnings and trip statistics are automatically updated.

1.5 Problem Definition

Many traditional transportation systems suffer from several challenges:

- Difficulty finding nearby drivers

- Long waiting time
- No live ride tracking
- Manual fare calculation
- Lack of ride history
- Poor communication between driver and passenger
- Limited driver management
- Weak security mechanisms
- No real-time notifications

These limitations reduce efficiency and customer satisfaction.

QuickRide addresses these problems by providing:

- Real-time ride booking
- Live location tracking
- Automatic fare calculation
- Secure authentication
- Instant ride notifications
- Online payment support
- Driver earnings management
- Ride history
- Rating and review system
- Cloud-based deployment

As a result, QuickRide offers a secure, reliable, and user-friendly ride-booking platform.



2. Technology and Literature review

2.1. About Tools and Technology

2.2. Brief History of Work Done



2.1 About Tools and Technology

QuickRide is a full-stack MERN (MongoDB, Express.js, React.js, and Node.js) based ride-booking application developed to provide users with a secure, fast, and real-time transportation service. The project uses modern web technologies and cloud services to ensure scalability, security, and smooth user experience. The application follows the MVC (Model–View–Controller) architecture, which separates the application's logic, user interface, and database operations, making the system easier to maintain and extend.

The frontend of QuickRide is developed using **React.js** and **Vite**. React.js provides a component-based architecture that enables the development of reusable and interactive user interfaces. Vite is used as the build tool because it offers faster development, quick page loading, and Hot Module Replacement (HMR), which improves developer productivity. **Tailwind CSS** is used to design a responsive and modern user interface that works efficiently on desktops, tablets, and mobile devices. Smooth animations and transitions are implemented using **GSAP**, creating an attractive and user-friendly experience.

The backend is built using **Node.js** and **Express.js**. Node.js provides a fast, event-driven runtime environment capable of handling multiple client requests simultaneously. Express.js simplifies the creation of RESTful APIs, routing, middleware, and server-side request handling. The application uses **Axios** for communication between the frontend and backend, allowing secure data exchange through HTTP requests.

For data storage, **MongoDB Atlas** is used as the cloud database, while **Mongoose ODM** manages database schemas and interactions. The database stores user information, captain details, ride requests, ratings, reviews, payment records, and blacklisted authentication tokens. MongoDB's flexible document-oriented structure makes it suitable for managing the application's dynamic data.

Security is implemented using **JSON Web Token (JWT)** authentication and **bcrypt** password encryption. JWT provides secure authentication and authorization by verifying user identity before granting access to protected routes. Passwords are encrypted using bcrypt before being stored in the database, ensuring user credentials remain secure. **Express Validator** is used to validate user input and prevent invalid or malicious data from entering the system.

One of the most important technologies used in QuickRide is **Socket.IO**, which enables real-time communication between passengers and drivers. It supports live ride requests, instant notifications, ride acceptance, live driver location updates, ride status synchronization, and ride completion events. This real-time functionality significantly improves the overall user experience.

The application integrates the **OpenRouteService Maps API** to obtain pickup and destination coordinates, calculate routes, estimate travel distance and time, and display navigation information. Fare calculation is based on the distance and travel time returned by the map service. The payment module is integrated with **Razorpay**, allowing users to complete secure online payments after the ride.

Finally, the frontend is deployed on **Vercel**, while the backend is hosted on **Render**, with **MongoDB Atlas** providing secure cloud-based database services. These technologies together create a reliable, scalable, and efficient ride-booking platform capable of serving multiple users simultaneously.

2.2 Brief History of Work Done

The development of the QuickRide Ride Booking System was carried out in a systematic manner, beginning with requirement analysis and ending with deployment. Initially, a detailed study of existing ride-booking platforms such as Uber and Ola was conducted to understand their features, workflow, advantages, and limitations. Based on this analysis, the project requirements were identified, including user registration, captain management, ride booking, real-time tracking, fare calculation, secure authentication, online payment, and rating systems.

During the first phase of development, the project architecture and database design were prepared using the MVC architecture. MongoDB Atlas was selected as the cloud database, while Node.js and Express.js were chosen for backend development. Database collections for users, captains, rides, ratings, and blacklisted tokens were designed to efficiently manage application data.

In the second phase, secure authentication was implemented using JWT and bcrypt. Separate registration and login modules were developed for users and captains, along with protected routes to prevent unauthorized access. Profile management, logout functionality, and token blacklisting were also implemented to improve system security.

The third phase focused on developing the ride management system. Users were provided with options to select pickup and destination locations, calculate ride fares, and book rides. Captains were given the ability to register vehicle details, switch between online and offline modes, receive ride requests, and accept or reject rides. Socket.IO was integrated to enable real-time communication between users and captains, allowing instant ride notifications, live location sharing, and ride status updates.

Finally, the application was thoroughly tested to ensure correct functionality, security, and performance. After successful testing, the frontend was deployed on **Vercel**, the backend on **Render**, and the database on **MongoDB Atlas**. The completed QuickRide application provides a secure, responsive, and real-time ride-booking platform that demonstrates the practical implementation of the MERN stack along with modern web technologies

3. System Requirement and study

- 3.1 User Characteristics
- 3.2 Hardware and Software Requirements
- 3.3 Constraints
 - 3.3.1 Regulatory Policies
 - 3.3.2 Hardware Limitations
 - 3.3.3 Parallel Operations
 - 3.3.4 Higher Order Languages Requirements
 - 3.3.5 Reliability Requirements
 - 3.3.6 Criticality of the Applications
 - 3.3.7 Safety and Security Consideration
- 3.4 Assumptions and Dependencies



3.1 User Characteristics

The QuickRide Ride Booking System is designed to be used by two primary types of users: **Passengers (Users)** and **Captains (Drivers)**. The application is developed with a simple and user-friendly interface so that people with basic knowledge of smartphones or computers can easily use the system.

Passenger (User)

Passengers are individuals who use the application to book rides. They can register using their personal details, log in securely, search for pickup and destination locations, view estimated fares, book rides, track the driver's live location, complete online payments, and provide ratings and reviews after the ride. The application requires only basic internet and smartphone knowledge, making it suitable for users of all age groups.

Captain (Driver)

Captains are registered drivers who provide transportation services through the QuickRide platform. They can create an account, register their vehicle details, switch their availability status between online and offline, receive ride requests, accept or reject bookings, navigate to passenger locations, complete rides, and monitor their earnings and ratings. The application is designed to be easy to operate, allowing drivers to manage rides efficiently.

Administrator (Future Enhancement)

Although the current version does not include a dedicated administrator panel, the system can be extended in the future with an admin module. The administrator will be able to manage users, captains, rides, payments, reports, and overall system activities through a centralized dashboard.

The application is designed to support users with different levels of technical knowledge while maintaining simplicity, efficiency, and reliability.

3.2 Hardware and Software Requirements

The successful operation of the QuickRide application requires both hardware and software resources. These requirements ensure smooth execution of frontend, backend, and database operations.

Hardware Requirements

The minimum hardware requirements for developing and running the QuickRide application are:

- Processor: Intel Core i3 or higher
- RAM: Minimum 8 GB (16 GB recommended)

- Hard Disk: 256 GB SSD or higher
- Display: 1366 × 768 resolution or above
- Internet Connection: Broadband connection
- Smartphone: Android or iOS device for testing

Software Requirements

The software used during the development and deployment of QuickRide includes:

- Operating System: Windows 10/11
- Frontend: React.js, Vite, Tailwind CSS
- Backend: Node.js, Express.js
- Database: MongoDB Atlas
- Programming Language: JavaScript
- IDE: Visual Studio Code
- Browser: Google Chrome or Microsoft Edge
- Version Control: Git and GitHub
- API Testing: Postman
- Hosting Platform: Vercel and Render
- Maps API: OpenRouteService
- Payment Gateway: Razorpay
- Package Manager: npm

These technologies provide a secure, scalable, and efficient environment for developing and deploying the application.

3.3 Constraints

The development and implementation of the QuickRide Ride Booking System are influenced by several technical, operational, and security constraints. These constraints define the boundaries within which the application must function while ensuring reliability and performance.

3.3.1 Regulatory Policies

QuickRide follows standard software development practices and security guidelines to protect user information and maintain data privacy. User passwords are encrypted using bcrypt before storage, and

authentication is managed using JWT tokens. The application validates user inputs to prevent unauthorized access and malicious requests.

The payment module is integrated using Razorpay, which follows secure payment standards for processing online transactions. All communication between the client and server is performed through secure REST APIs.

The system also respects user privacy by storing only the necessary information required for ride booking and management.

3.3.2 Hardware Limitations

The performance of the application depends on the available hardware resources and internet connectivity.

Older devices with limited memory or slower processors may experience reduced performance while running the application. Since QuickRide uses real-time communication and live location updates, a stable internet connection is required for accurate ride tracking.

Similarly, the server hosting the backend should have sufficient processing power and network bandwidth to handle multiple concurrent users and ride requests efficiently.

Despite these limitations, the application is optimized to run smoothly on most modern computers and smartphones.

3.3.3 Interfaces to Other Applications

The **QuickRide Ride Booking System** interacts with several external applications and services to provide complete functionality. It integrates with the **OpenRouteService Maps API** to obtain location coordinates, calculate routes, estimate travel distance, and display navigation information. The system also uses **Razorpay** for secure online payment processing. **MongoDB Atlas** serves as the cloud database for storing user, captain, ride, and payment information, while **Socket.IO** enables real-time communication between passengers and captains for ride requests, live location updates, and ride status notifications. These integrations improve the efficiency, reliability, and overall user experience of the QuickRide application.

3.3.4 Parallel Operations

QuickRide supports multiple users and captains using the system simultaneously.

Several operations can occur in parallel, including:

- Multiple users booking rides at the same time.
- Multiple captains receiving ride requests.

- Live location updates from different drivers.
- Real-time ride status synchronization.
- Simultaneous payment processing.
- Concurrent database operations.

Node.js and Socket.IO efficiently handle these parallel requests using asynchronous programming techniques, ensuring fast response times and reliable system performance.

3.3.5 Higher Order Language Requirements

QuickRide is developed entirely using modern high-level programming languages and frameworks.

The primary programming language is **JavaScript**, which is used for both frontend and backend development.

The technologies used include:

- JavaScript (ES6+)
- React.js
- Node.js
- Express.js
- Tailwind CSS
- HTML5
- CSS3

These languages and frameworks simplify application development, improve maintainability, and support rapid implementation of new features.

3.3.6 Reliability Requirements

Reliability is one of the most important aspects of the QuickRide application.

The system should provide uninterrupted services during ride booking and communication between passengers and drivers.

To ensure reliability, the application includes:

- Secure user authentication
- Error handling mechanisms

- Database validation
- Automatic fare calculation
- Stable Socket.IO communication
- Protected REST APIs
- Cloud database backup through MongoDB Atlas

These features help maintain accurate ride information and minimize application failures.

3.3.7 Criticality of the Application

QuickRide is a real-time transportation application where delays or failures may directly affect passengers and drivers.

Critical operations include:

- User authentication
- Ride booking
- Captain matching
- Live location sharing
- Fare calculation
- Online payment
- Ride completion

Any failure in these modules may interrupt the ride booking process. Therefore, these operations require high accuracy, proper validation, and reliable communication between the client and server.

3.3.8 Safety and Security Consideration

Security is an essential requirement of the QuickRide application.

The following measures have been implemented to ensure data safety and secure communication:

- JWT-based authentication
- Password encryption using bcrypt
- Token blacklisting after logout
- Protected API routes
- Input validation using Express Validator
- Secure database storage in MongoDB Atlas
- Environment variables for storing confidential API keys
- HTTPS deployment on Vercel and Render
- Authentication middleware for authorized access

These security mechanisms protect user information, prevent unauthorized access, and improve the overall reliability of the system.

3.4 Assumptions and Dependencies

The successful operation of QuickRide depends on several assumptions and external dependencies. The application assumes that users and captains have smartphones or computers with internet connectivity. It also assumes that GPS and location services are enabled to provide accurate pickup, destination, and live tracking information.

The project depends on several third-party services, including MongoDB Atlas for cloud database storage, OpenRouteService Maps API for route calculation and navigation, Razorpay for online payment processing, Render for backend hosting, and Vercel for frontend deployment.

The application also assumes that these external services remain available and functional during normal operation. Any temporary service outage may affect features such as live location tracking, payment processing, or route calculation. However, the modular architecture of QuickRide allows these services to be replaced or upgraded in the future with minimal changes to the application.

4. System Analysis

4.1 Study of Current System

4.2 Problems and Weaknesses of Current System

4.3 Requirements of New System

4.3.1 User Requirements

4.3.2 System Requirements

4.4 Feasibility Study

4.4.1 Does the system contribute to the overall objectives of the organization?

4.4.2 Can the system be implemented using the current technology and within the given cost and schedule constraints?

4.4.3 Can the system be integrated with other systems that are already in place?

4.5 Requirement Validation

4.6 Activity / Process in New System

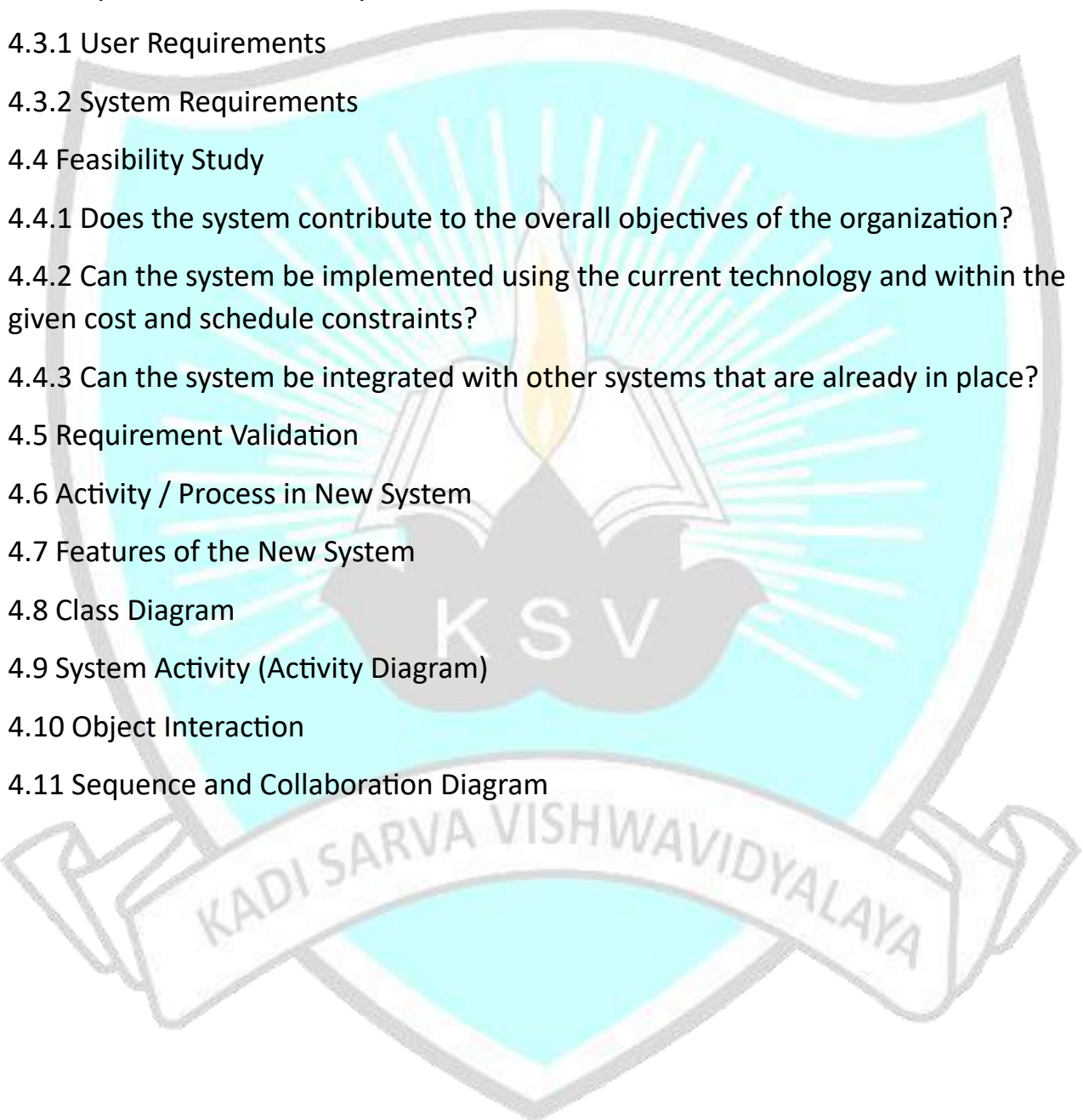
4.7 Features of the New System

4.8 Class Diagram

4.9 System Activity (Activity Diagram)

4.10 Object Interaction

4.11 Sequence and Collaboration Diagram



4.1 Study of Current System

The existing transportation and taxi booking system mainly depends on traditional methods such as phone calls, local taxi stands, or manual booking services. In many cities, passengers need to contact drivers directly or wait at taxi stands to find available vehicles. This process is often time-consuming and inconvenient, especially during peak hours or emergencies. Users usually do not know the exact arrival time of the driver, estimated fare, or vehicle availability before booking.

Some existing ride-booking applications provide online services, but many of them are expensive, have limited coverage, or do not offer sufficient real-time features for small transportation businesses. In some cases, passengers experience delays because there is no proper communication between drivers and users. Manual fare calculation may also result in incorrect pricing and disputes between passengers and drivers.

Traditional systems generally lack proper ride history, driver ratings, online payment options, and secure authentication mechanisms. Many local taxi services also do not provide live location tracking, making it difficult for passengers to monitor their rides. Drivers face challenges in managing their rides, earnings, and availability due to the absence of a centralized platform.

The current system also has limited automation. Ride requests are not processed efficiently, and matching passengers with nearby drivers often requires manual effort. These limitations reduce operational efficiency and affect the overall customer experience.

QuickRide is designed to overcome these issues by providing a secure, real-time, and user-friendly ride-booking platform. The system automates ride booking, driver matching, fare calculation, and ride tracking while ensuring secure communication between passengers and captains.

4.2 Problem and Weaknesses of Current System

The traditional ride-booking system has several problems that reduce its efficiency and reliability. One of the major problems is the lack of real-time communication between passengers and drivers. Since ride requests are handled manually, users often experience long waiting times and delays in finding available vehicles.

Another weakness is the absence of live ride tracking. Passengers cannot monitor the driver's location or estimate the arrival time accurately. This reduces transparency and creates inconvenience during travel.

Manual fare calculation is another limitation. Different drivers may charge different fares for the same route, leading to confusion and customer dissatisfaction. The absence of automatic fare calculation also increases the possibility of pricing errors.

Security is another concern in traditional systems. Many local taxi services do not provide secure user authentication or encrypted password storage. User information may not be properly protected, increasing the risk of unauthorized access.

The current system also lacks online payment facilities, requiring passengers to make cash payments. There is no centralized record of completed rides, making it difficult for users to access ride history or previous transactions.

4.3 Requirements of New System

The proposed QuickRide Ride Booking System is developed to overcome the limitations of the existing transportation system. The new system provides secure authentication, automated ride booking, real-time communication, live location tracking, online payment integration, and efficient ride management.

The system requirements are divided into user requirements and system requirements to ensure that both user expectations and technical requirements are satisfied.

4.3.1 User Requirements

The QuickRide application should satisfy the following user requirements:

- Users should be able to register and log in securely.
- Captains should be able to register with their vehicle details.
- Passengers should be able to search pickup and destination locations easily.
- The system should calculate ride fares automatically based on travel distance.
- Users should receive real-time ride status updates.
- Captains should receive instant ride request notifications.
- Passengers should be able to track the driver's live location.
- Users should have access to their ride history.
- Online payment should be available through Razorpay.
- Passengers should be able to rate drivers and submit reviews after ride completion.
- Captains should be able to monitor their earnings and completed trips.
- The application should provide a responsive interface for both desktop and mobile devices.

These requirements ensure a smooth and convenient ride-booking experience for both passengers and captains.

4.3.2 System Requirements

The QuickRide system should satisfy the following technical requirements:

- The frontend should be developed using React.js and Tailwind CSS.
- The backend should be developed using Node.js and Express.js.
- MongoDB Atlas should be used for cloud database storage.
- The application should follow the MVC architecture.
- JWT should be implemented for secure authentication.
- Passwords should be encrypted using bcrypt.
- Socket.IO should provide real-time communication.
- OpenRouteService Maps API should calculate routes, distance, and travel time.
- Razorpay should support secure online payments.
- REST APIs should be protected using authentication middleware.
- User inputs should be validated using Express Validator.
- The application should support multiple users simultaneously.
- Error handling and logging mechanisms should be implemented for reliability.

These technical requirements ensure that the system remains secure, scalable, and efficient.

4.4 Feasibility Study

A feasibility study is conducted to determine whether the QuickRide Ride Booking System can be successfully developed and implemented using the available technology, budget, and development schedule.

The study evaluates the project from organizational, technical, economic, and integration perspectives.

4.4.1 Contribution of the System to Organizational Objectives

Yes. The QuickRide Ride Booking System supports the primary objective of providing a secure, efficient, and reliable transportation platform.

The system automates ride booking, driver matching, fare calculation, payment processing, and ride management. It reduces manual work and improves communication between passengers and drivers.

Real-time tracking and secure authentication increase customer satisfaction and service quality. The rating and review system helps maintain service standards while the captain dashboard allows drivers to monitor their performance and earnings.

Overall, the proposed system improves operational efficiency and supports future expansion of transportation services.

4.4.2 Feasibility within Current Technology, Cost, and Schedule Constraints

Yes. The QuickRide system can be developed using widely available open-source technologies such as React.js, Node.js, Express.js, MongoDB Atlas, and Socket.IO.

Most development tools used in the project are free or provide generous free tiers, reducing the overall project cost. Services such as Vercel, Render, and MongoDB Atlas allow cloud deployment without significant infrastructure investment.

The modular MVC architecture simplifies development, testing, and maintenance, allowing the project to be completed within the planned development schedule.

Therefore, the project is technically, economically, and practically feasible.

4.4.3 Integration with Existing Systems

Yes. QuickRide is designed using REST APIs and standard JSON data formats, making integration with other software systems straightforward.

The application can easily integrate with:

- Online payment gateways
- GPS and navigation services
- Third-party authentication systems
- Notification services
- Fleet management systems
- Administrative dashboards
- Future mobile applications

The modular architecture also allows future enhancements without affecting the existing system.

4.5 Requirements Validation

Requirement validation ensures that the developed QuickRide system satisfies all user and system requirements correctly.

Validation was carried out by comparing the implemented modules with the documented project requirements. User registration and login were tested to verify secure authentication using JWT and bcrypt. Ride booking functionality was validated to ensure that users could successfully select pickup and destination locations, calculate fares, and book rides.

The real-time communication module was tested to confirm that ride requests, driver acceptance, location updates, and ride completion notifications were delivered instantly using Socket.IO. Online payment functionality was verified using the Razorpay integration, while the rating and review system was tested after successful ride completion.

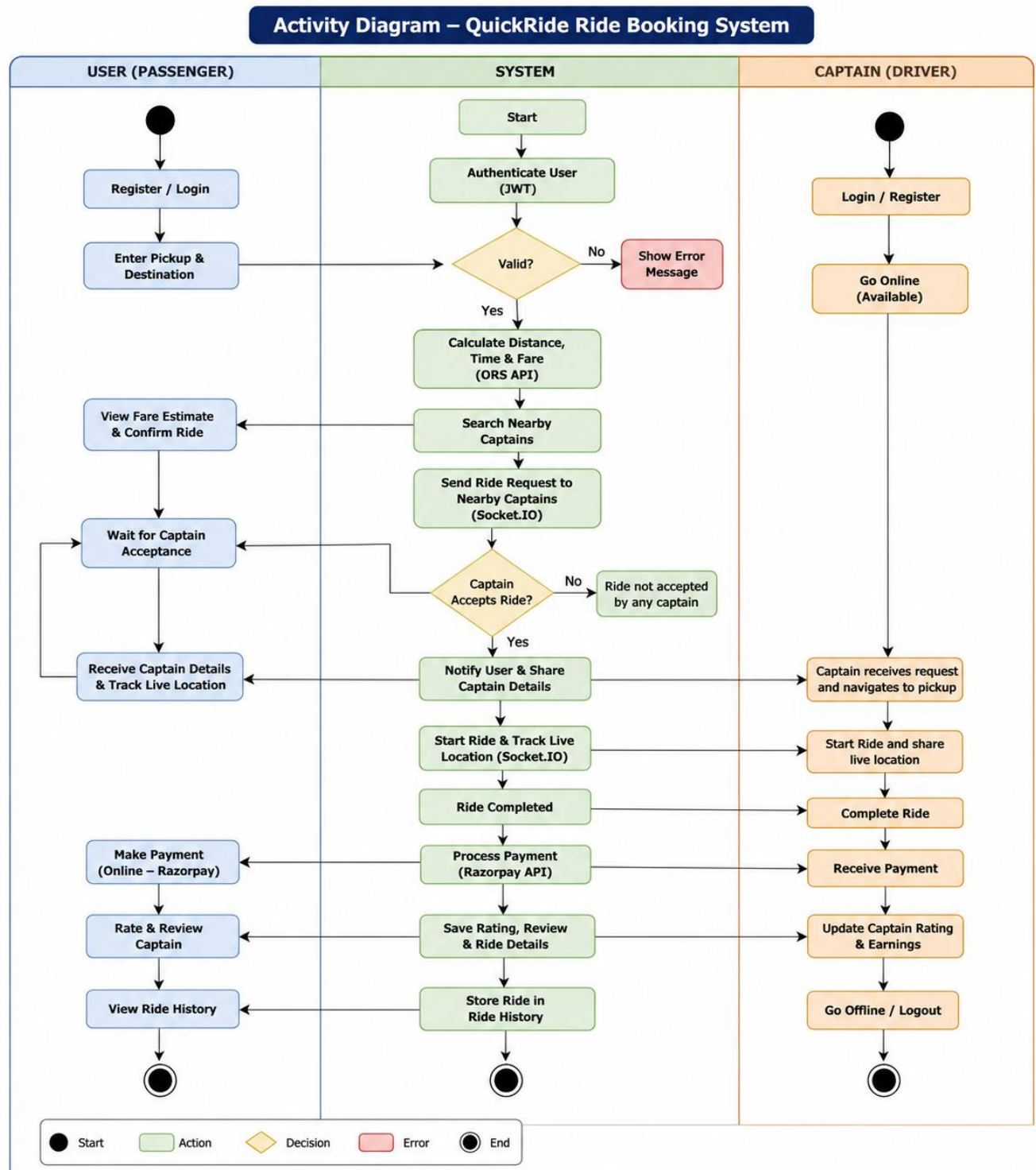
The application was also tested for input validation, error handling, database operations, and security features. Multiple test cases confirmed that the system performs accurately, securely, and efficiently under normal operating conditions.

The validation process demonstrates that QuickRide satisfies the functional and technical requirements of a modern ride-booking application and is suitable for practical implementation.



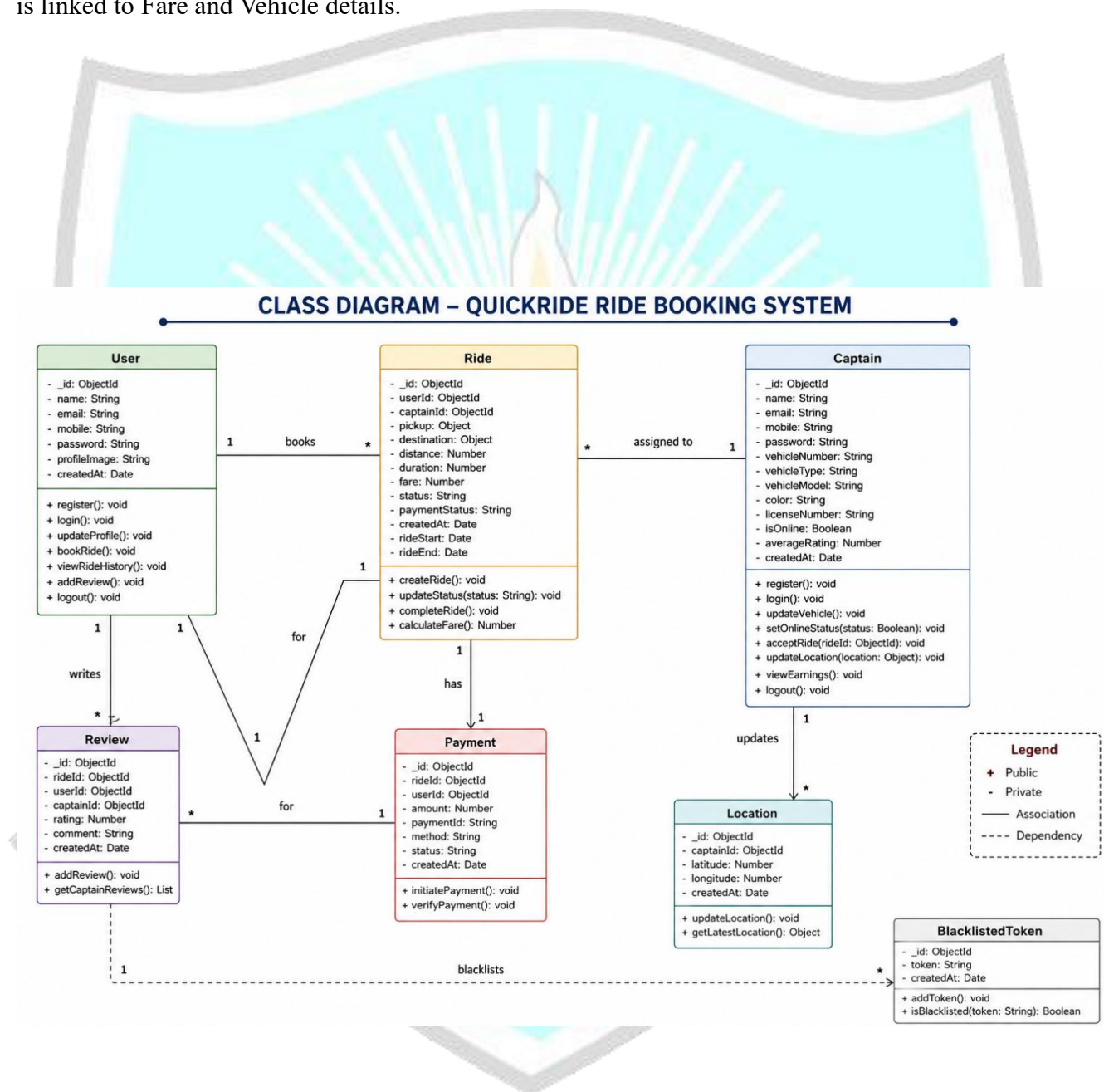
1. Activity Diagram

The activity diagram shows the ride booking flow: user logs in, enters pickup and destination, and the system calculates fare and shows vehicle options. Once confirmed, a captain is matched, accepts the ride, and completes the trip ending in payment.



2. Class Diagram

Shows core entities — User, Captain, Ride, and Vehicle — with attributes like name, email, password, location, and vehicleType. Relationships include User places a Ride, Captain accepts a Ride, and Ride is linked to Fare and Vehicle details.

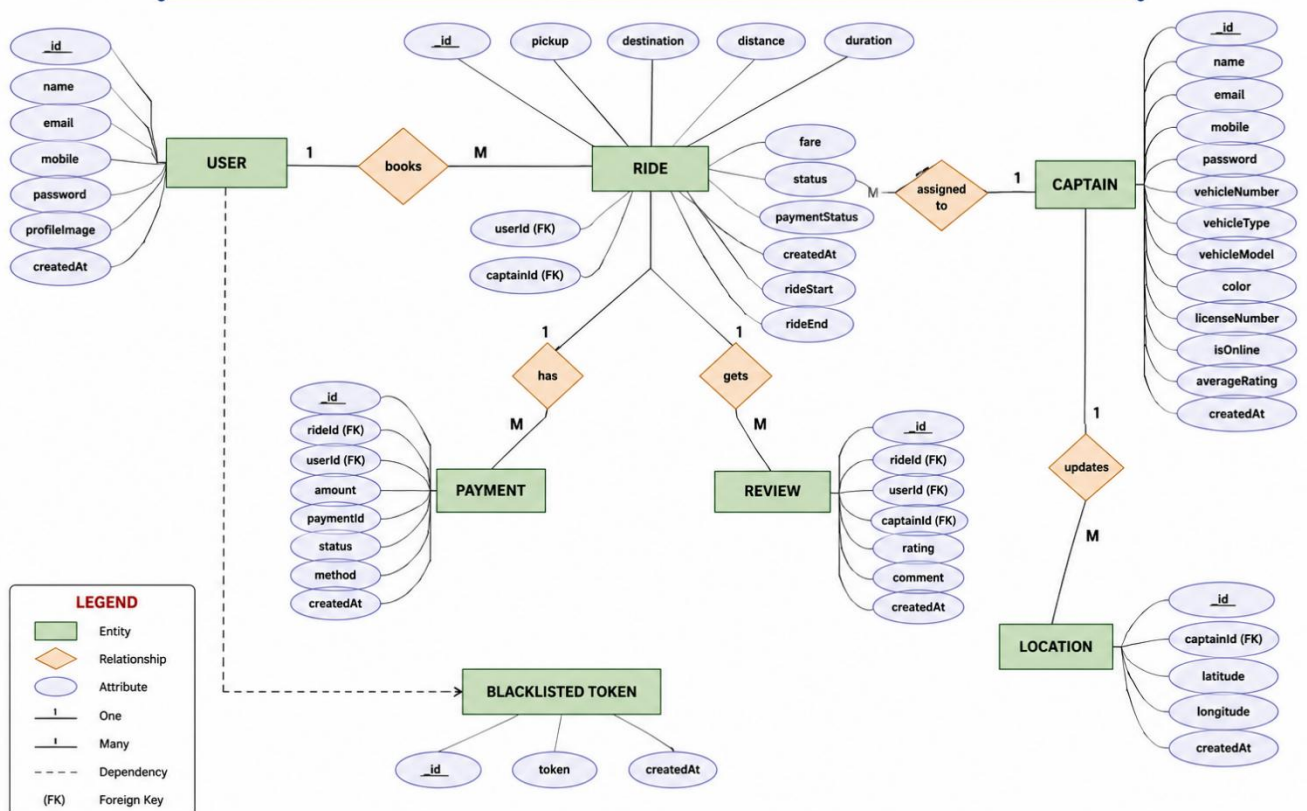


3. E-R Diagram

Represents entities like User, Captain, Ride, and Vehicle with their attributes and relationships. A User can request multiple Rides, each Ride is assigned to one Captain, and each Ride is associated with pickup, destination, and fare details.

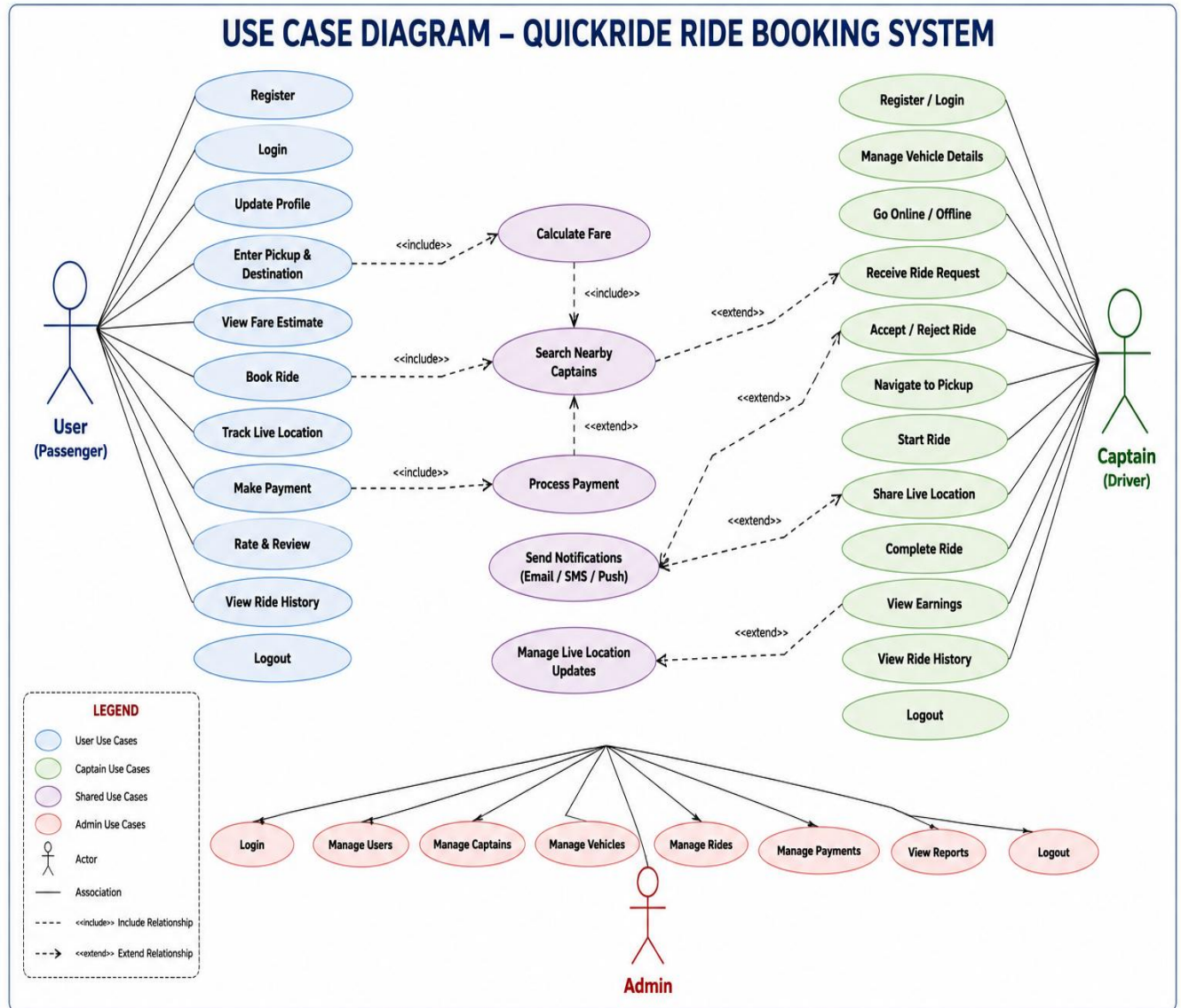


ER DIAGRAM – QUICKRIDE RIDE BOOKING SYSTEM



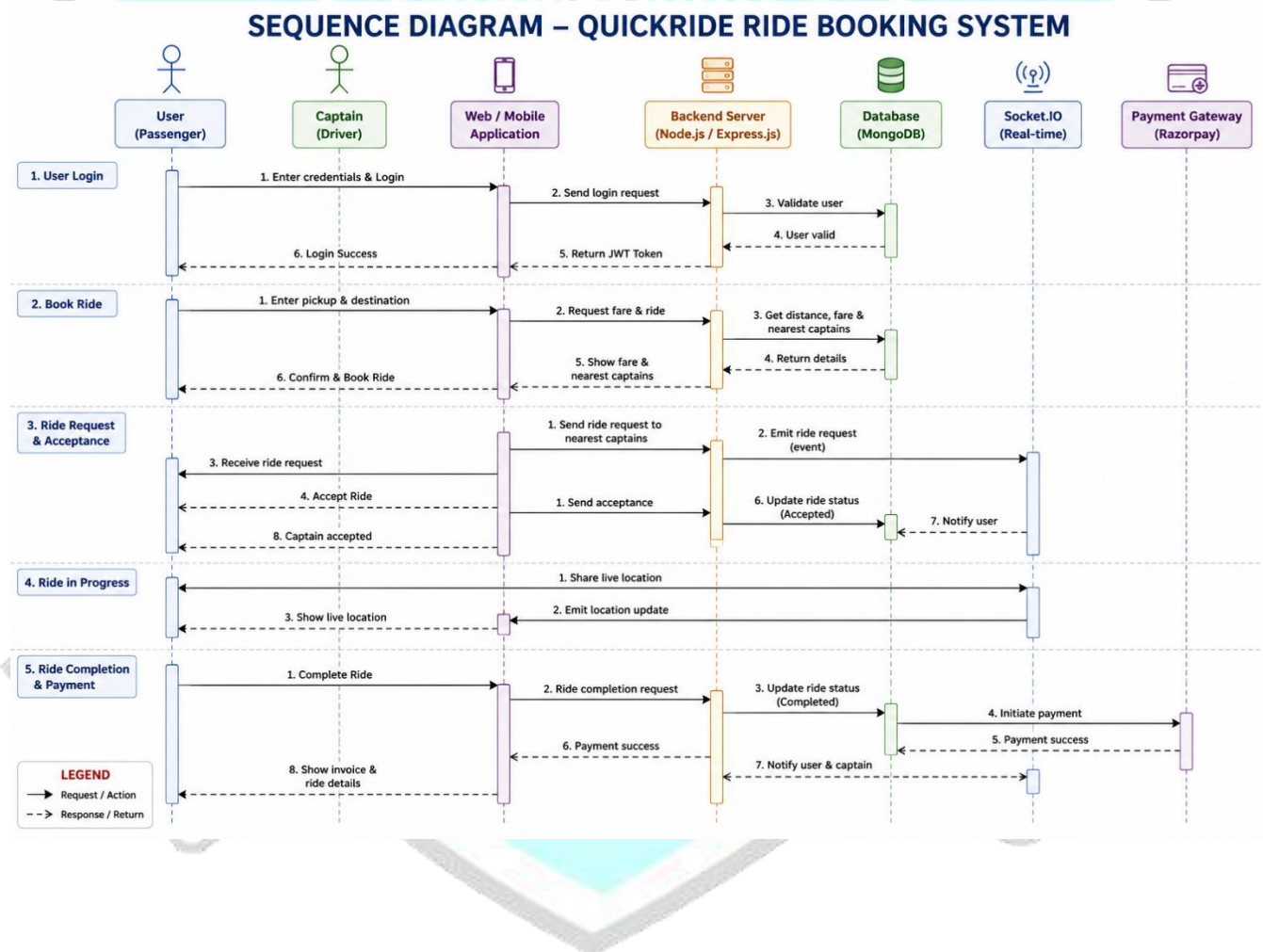
4. System (Use Case) Diagram

Includes actors User and Captain. User use cases: Register/Login, Search Location, Book Ride, Make Payment. Captain use cases: Register/Login, Accept Ride, Navigate to Pickup, Complete Ride.



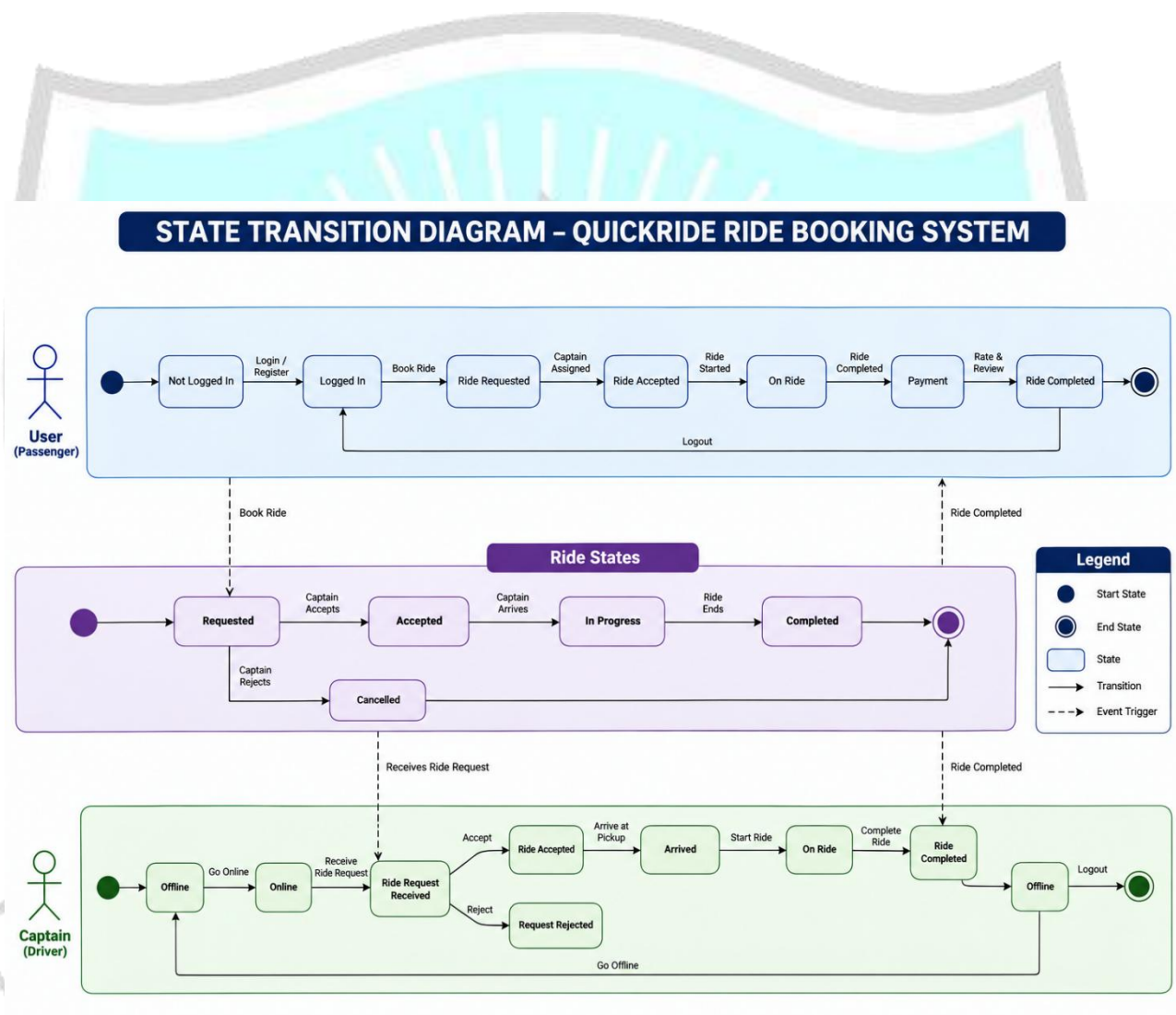
5. Object Interaction (Sequence Diagram)

Shows interaction between User, Frontend, Backend, and Captain during ride booking: User sends location → Backend calculates fare → Backend notifies Captain → Captain accepts → Ride starts → Ride completes with payment confirmation.



6. State Transition Diagram

Represents the ride lifecycle: Requested → Accepted → Ongoing → Completed (or Cancelled at any stage before completion), showing how a ride moves between these states based on user and captain actions.



5. System Design

5.1 System application Design

5.1.1 Method Pseudo code

5.2 Database Design/Data Structure Design

5.2.1 Table and Relationship

5.2.2 Logical Description of Data

5.3 Input/Output and Interface Design

5.3.1 State Transition/UML Diagram

5.3.2 Sample of Forms, Reports and Interface



5.1 System Application Design

The **QuickRide Ride Booking System** is designed using the **Model–View–Controller (MVC)** architecture to ensure modularity, maintainability, and scalability. The system consists of three major components: the frontend, backend, and cloud database. The frontend is developed using **React.js** with **Vite** and **Tailwind CSS**, providing a responsive and user-friendly interface for passengers and captains. The backend is developed using **Node.js** and **Express.js**, which handle API requests, authentication, ride management, payment processing, and real-time communication. **MongoDB Atlas** is used as the cloud database for storing application data.

The application follows a client-server architecture where the frontend communicates with the backend through secure REST APIs. User authentication is implemented using **JWT (JSON Web Token)** and passwords are encrypted using **bcrypt** before storage. Real-time communication between passengers and captains is handled using **Socket.IO**, enabling instant ride requests, live driver location updates, ride status notifications, and ride completion events.

The system integrates the **OpenRouteService Maps API** for obtaining pickup and destination coordinates, calculating travel distance and estimated time, and displaying routes. **Razorpay** is integrated to support secure online payments after ride completion.

The overall system design includes the following modules:

- User Registration and Login
- Captain Registration and Login
- Ride Booking Module
- Fare Calculation Module
- Live Location Tracking
- Ride Management
- Rating and Review System
- Payment Module
- Captain Dashboard
- User Dashboard
- Ride History
- Authentication and Security

The modular design allows future enhancements without affecting existing functionalities and supports efficient maintenance and scalability.

5.1.1 Method Pseudo Code

Function: User Registration

START

Enter user details

Validate input

If details are valid

Encrypt password

Store user information in database

Generate JWT Token

Return Success

Else

Return Error

END

Function: Ride Booking

START

User enters pickup location

User enters destination

Calculate route distance

Calculate estimated fare

Search nearest available captain

Send ride request

Wait for captain response

If accepted

Confirm ride

Else

Search next captain

END

Function: Captain Ride Acceptance

START

Captain receives notification

Display ride details

Accept or Reject request

If accepted

Update ride status

Share live location

Navigate to passenger

Else

Notify user

END

5.2 Database Design / Data Structure Design

The QuickRide application uses **MongoDB Atlas**, a cloud-based NoSQL database, to store application data securely and efficiently. MongoDB provides flexible document-based storage that is suitable for handling user information, ride details, captain data, ratings, and payment records.

Each collection stores related information and maintains relationships using Object IDs. The database is designed to ensure data consistency, scalability, and high performance.

The major collections in the database are:

- User Collection
- Captain Collection
- Ride Collection
- Rating Collection
- Blacklisted Token Collection

The database supports CRUD operations, secure authentication, ride management, payment processing, and reporting.

.

5.2.1 Tables and Relationships

Although MongoDB uses collections instead of traditional tables, the logical structure is similar to relational databases.

User Collection

Fields:

- User ID
- Name
- Email
- Password
- Mobile Number
- Profile Image
- Created Date

Relationship:

- One user can book multiple rides.
- One user can submit multiple ratings and reviews.

Captain Collection

Fields:

- Captain ID
- Name
- Email
- Password
- Vehicle Name
- Vehicle Number
- Vehicle Type
- Online Status
- Average Rating

Relationship:

- One captain can complete multiple rides.
- One captain receives multiple ratings.

Ride Collection

Fields:

- Ride ID
- User ID

- Captain ID
- Pickup Location
- Destination Location
- Distance
- Fare
- Ride Status
- Payment Status
- Ride Date

Relationship:

- Each ride belongs to one user.
- Each ride is assigned to one captain.

Rating Collection

Fields:

- Rating ID
- Ride ID
- User ID
- Captain ID
- Overall Rating
- Comfort Rating
- Safety Rating
- Price Rating
- Driver Behaviour Rating
- Review

Relationship:

- One completed ride can have one review.
- One captain can receive multiple reviews.

Blacklisted Token Collection

Fields:

- Token ID
- JWT Token
- Created Date

Relationship:

- Used during logout to prevent unauthorized access using old authentication tokens.

1. Users Table

Field	Type	Required	Default
_id (Primary Key)	ObjectId	Yes	Auto Generated
fullname.firstname	String	Yes	—
fullname.lastname	String	No	—
email	String (Unique)	Yes	—
password	String (Hashed)	Yes	—
socketId	String	No	Null
createdAt	Timestamp	Yes	Current Time

2. Resume Table

Field	Type	Required	Default
_id (Primary)	ObjectId	Yes	auto
userId (Foreign)	ObjectId	Yes	—
fileUrl	String	Yes	—
extractedText	String	Yes	—

fileType	String	Yes	pdf/docx
uploadedAt	Timestamp	Yes	now



3. Admin Table

Field	Type	Required	Default
_id (primary)	ObjectId	Yes	auto
name	string	Yes	—
email	string	Yes	—
password	string	Yes	—
role	string	Yes	admin
createdAt	timestamp	Yes	now

4. Authentication & User Management Table

Field	Type	Required	Default
_id (Primary)	ObjectId	Yes	auto
userId (Foreign)	ObjectId	Yes	—
jwtToken	String	Yes	—
loginAt	Timestamp	Yes	now
tokenExpire	Timestamp	Yes	JWT Expiry Time

isActive	boolean	Yes	true
----------	---------	-----	------

5. Features Table

Field	Type	Required	Default
_id (primary)	ObjectId	Yes	auto
featureName	string	Yes	—
description	string	Yes	—
isEnabled	boolean	Yes	true
createdAt	timestamp	Yes	now

6. QuickRide Table

Field	Type	Required	Default
_id (Primary)	ObjectId	Yes	auto
userId (Foreign)	ObjectId	Yes	—
captainId (Foreign)	ObjectId	Yes	—

pickupLocation	String	Yes	—
destinationLocation	String	Yes	—
fare	Number	Yes	0
rideStatus	String	Yes	pending
createdAt	Timestamp	Yes	now

7. Integration & Performance Table

Field	Type	Required	Default
_id (primary)	ObjectId	Yes	auto
serviceName	string	Yes	—
apiStatus	string	Yes	Active
responseTime	number	Yes	—
errorMessage	string	No	null
loggedAt	timestamp	Yes	now

8. Security & UI Table

Field	Type	Required	Default
_id (primary)	ObjectId	Yes	auto
userId (foreign)	ObjectId	Yes	—
failedLoginCount	number	Yes	0
lastLoginIP	string	No	null
themePreference	string	Yes	light
createdAt	timestamp	Yes	now

5.3 Input / Output and Interface Design

The QuickRide application provides a simple, interactive, and responsive interface that enables users and captains to perform all operations efficiently. All inputs are validated on both the client side and server side before processing.

Input Design

The application accepts the following inputs:

- User Registration Details
- Captain Registration Details
- Login Credentials
- Pickup Location
- Destination Location
- Vehicle Information
- Payment Details
- Rating and Review
- Profile Information

Each input field is validated to ensure correct format and prevent invalid data entry.

Output Design

The system generates the following outputs:

- Successful Registration Message
- Login Confirmation
- Estimated Fare
- Ride Confirmation
- Live Driver Location
- Ride Status Updates
- Payment Confirmation
- Ride History
- Captain Earnings
- Rating Summary
- Error and Warning Messages

Outputs are displayed in a clean and organized manner, allowing users to understand application responses easily.

Interface Design

The user interface is developed using **React.js** and **Tailwind CSS**, ensuring a modern, responsive, and mobile-friendly experience.

The main interfaces include:

- Home Page
- Login Page
- Registration Page
- User Dashboard
- Captain Dashboard
- Ride Booking Screen
- Ride Tracking Screen
- Payment Screen
- Rating Screen
- Profile Screen
- Ride History Screen

Animations created using GSAP improve user interaction and navigation.

5.3.1 State Transition / UML Diagram

The State Transition Diagram represents different states of the QuickRide application and the transitions between them based on user actions.

Major States

- Initial State
- User Registration
- User Login
- Captain Login
- Ride Booking
- Searching Captain
- Ride Accepted
- Ride Started
- Ride Completed
- Payment Completed
- Rating Submitted
- Logout

State Transitions

- Start → Register/Login
- Login → Dashboard
- Dashboard → Book Ride
- Book Ride → Searching Captain
- Searching Captain → Ride Accepted
- Ride Accepted → Ride Started
- Ride Started → Ride Completed
- Ride Completed → Payment
- Payment → Rating
- Rating → Logout

The State Transition Diagram helps understand how the system moves from one operation to another while maintaining data consistency.

5.3.2 Samples of Forms, Reports, and Interface

The QuickRide application contains several forms and interfaces that provide smooth interaction between users and the system.

Forms

- User Registration Form
- Captain Registration Form

- Login Form
 - Ride Booking Form
 - Payment Form
 - Rating and Review Form
 - Profile Update Form
-

Reports

The application generates several useful reports, including:

- Ride History Report
 - Captain Earnings Report
 - User Booking Report
 - Rating and Review Report
 - Payment Summary Report
-

Interface Screens

The application includes the following interface screens:

- Welcome Screen
- User Login Screen
- Captain Login Screen
- Registration Screen
- Home Dashboard
- Ride Booking Interface
- Driver Tracking Screen
- Payment Interface
- Rating Interface
- Captain Dashboard
- User Profile
- Ride History Screen

The interface is designed to be responsive, attractive, and easy to use on both desktop and mobile devices. Clear navigation, modern UI components, and smooth animations ensure a better user experience throughout the application.

7. Snapshot



Get Started with QuickRide

Continue

Where to?

Book a ride in seconds



Gandhinagar, GJ, India



Thaltej, GJ, India

Find Ride



Search Location

Searching for destination



Thaltej, Ahmadabad, GJ, India



Tap to select this location



Thaltej, GJ, India



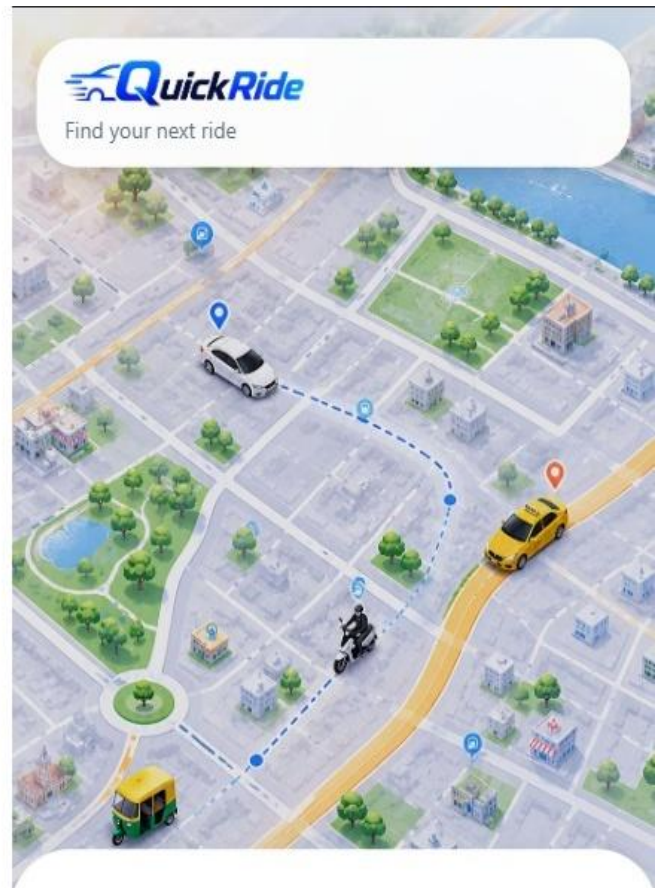
Tap to select this location



Thaltej Road, Thaltej,



Find your next ride



Where to?

Book a ride in seconds



Enter pickup location



Where are you going?



Find your next ride

Choose a Ride

Select the vehicle that suits your trip



UberGo 4

2 min away
Affordable everyday rides

₹433



Moto 1

3 min away
Fast and affordable bike rides

₹171



Uber Auto 3

2 min away
Budget-friendly auto rides

₹277

Driver Assigned

Live

Share OTP with your driver to start the ride

Ride OTP

903289



PICKUP

Gandhinagar, GJ, India



DESTINATION

Thaltej, GJ, India

Total Fare

₹171



Tell the OTP to your driver only after they arrive at your pickup location.

